Complete Project Documentation: Distributed Federated Learning Network

1. Project Abstract & Motivation

In the modern healthcare ecosystem, protecting patient privacy while leveraging data for intelligent decision-making is a critical challenge. Hospitals often possess highly valuable diagnostic data but cannot legally share it due to strict privacy laws (like HIPAA or India's DPDP Act).

The Solution: This project implements **Federated Learning (FL)**. Instead of moving sensitive patient data to a centralized server for AI training, this architecture leaves the data exactly where it is. The central server distributes an untrained Artificial Intelligence model to the hospitals. The hospitals train the model locally using their private data, and only the mathematically encrypted *model weights* are sent back to the central server.

Originally conceived as a monolithic Django application, this project was architecturally upgraded into a **Polyglot Microservices Network** using Java, Spring Boot, Apache Kafka, and Python to handle enterprise-grade scalability and asynchronous processing.

---

2. Technology Stack

- **Central Admin Node:** Java 21, Spring Boot 3.3, Spring Security, JWT (JSON Web Tokens)
- **Message Broker:** Apache Kafka (Embedded Zookeeper/Kafka Broker handling async streams)
- **Database:** Neon Serverless PostgreSQL (Cloud Database for tracking hospitals and models)
- **Machine Learning Edge Node:** Python 3.11, TensorFlow 2.x, Keras, Pandas, Kafka-Python
- **Testing / APIs:** Native PowerShell / CMD REST invocations (cURL, Invoke-RestMethod)

---

3. Architecture Breakdown

Service A: The Admin Orchestrator (Java / Spring Boot)

The Admin Node is the gatekeeper of the network. It handles:

1. **Hospital Onboarding:** Provides a `/api/auth/register` endpoint to save hospital credentials into the Neon DB.
2. **Security:** Provides a `/api/auth/login` endpoint that generates a secure, stateless JWT token.
3. **Kafka Broker:** Hosts an Embedded Kafka Broker on port `9092` with two dedicated channels:
    - `global-model-updates`: Used to broadcast the latest AI model.
    - `hospital-local-weights`: Used to collect trained weights from hospitals.

Service B: The Hospital ML Client (Python / TensorFlow)

The Python microservice represents an edge device sitting inside a hospital's private intranet.

1. **Kafka Listener:** Connects to port `9092` and continuously listens for the `global-model-updates` topic.
2. **Data Ingestion:** Loads `mock_patient_data.csv` (simulating secure, on-premise medical records).

3. **Deep Learning:** Upon receiving a Kafka signal, it spins up a 3-layer Sequential Neural Network (Dense 16 -> Dense 8 -> Dense 1) and trains it on the CSV data for 5 Epochs.
4. **Weight Publishing:** Extracts the accuracy metrics/weights and pushes them back to the `hospital-local-weights` Kafka topic.

---

4. Step-by-Step Setup & Run Guide

To run this distributed pipeline on a local machine, you must run both microservices simultaneously in separate terminal windows.

Step 1: Start the Java Admin Server

1. Open a terminal and navigate to the `admin-service` folder.
2. Run the Spring Boot application:

```
mvn spring-boot:run
```

3. Wait for the logs to indicate that Tomcat has started on port `8080` and the Embedded Kafka Broker is listening on `127.0.0.1:9092`.

Step 2: Start the Python ML Node

1. Open a **second** terminal and navigate to the `hospital-node` folder.
2. Activate the Python virtual environment and run the ML script:

```
.\venv\Scripts\activate
python main.py
```

3. The terminal will output `🎧 Listening for Global AI Model from Admin Node...` and patiently wait.

Step 3: Authenticate & Trigger the Federated Pipeline

1. Open a **third** terminal (PowerShell).
2. Register a hospital (only needed once):

```
Invoke-RestMethod -Uri "http://localhost:8080/api/auth/register" -Method Post
-ContentType "application/json" -Body '{"name": "Delhi City Hospital",
"email": "admin@delhicityhospital.in", "password": "securepassword123"}'
```

3. Log in to get the JWT token, and immediately pass that token to the `/api/fl/broadcast` endpoint:

```
$login = Invoke-RestMethod -Uri "http://localhost:8080/api/auth/login" -
Method Post -ContentType "application/json" -Body '{"email":
"admin@delhicityhospital.in", "password": "securepassword123"}'

$token = $login.token

Invoke-RestMethod -Uri "http://localhost:8080/api/fl/broadcast" -Method Post
-Headers @{ Authorization = "Bearer $token" }
```

Step 4: Observe the Microservice Communication

As soon as Step 3 executes:

1. The Java terminal will log that it successfully flushed a message to the Kafka broker.
2. The Python terminal will instantly react, bypassing standard HTTP routing and pulling directly from the Kafka stream.
3. TensorFlow will initialize and print the progress of 5 Neural Network training Epochs.
4. Python will automatically publish a success payload back into the Kafka stream.

---

5. API Reference Guide

Endpoint	Method	Security	Description
`/api/auth/register`	POST	Public	Registers a new hospital in the Neon PostgreSQL database.
`/api/auth/login`	POST	Public	Authenticates a hospital and returns a 24-hour JWT string.
`/api/fl/broadcast`	POST	Bearer JWT	Drops a "START_TRAINING" event into the Kafka Broker to wake up Python nodes.

---

6. Future Scope

- **GitHub Actions CI/CD:** Implement `.github/workflows` to automatically test the Java and Python builds whenever code is pushed.
- **Complex Aggregation:** Write a Kafka Consumer in the Java Admin node that listens to the `hospital-local-weights` topic, collects weights from multiple hospitals simultaneously, and runs a mathematical Federated Averaging (FedAvg) algorithm to create `Version 2.0` of the global model.